



UNIVERSIDAD TÉCNICA DE AMBATO



FACULTAD DE INGENIERIA EN SISTEMAS, ELECTRONICA E INDUSTRIAL

INFORME ACADEMICO

Nombre:

ESTIVEN SEBASTIAN CHILUISA VASQUEZ

Docente

JOSE RUBEN CAIZA CAIZABUANO

Materia:

Estructura de datos

Carrera:

Ing. Software

**Ambato - Ecuador
2026**

1. Introducción

El presente informe documenta el diseño e implementación de un sistema de gestión de personal en C++, fundamentado en la estructura de datos no lineal Árbol Binario de Búsqueda (BST). Esta estructura se seleccionó por su capacidad superior para organizar y recuperar información de manera jerárquica frente a las limitaciones de las estructuras lineales tradicionales.

A diferencia de los arreglos o listas (con complejidad $O(n)$), el BST optimiza las operaciones de inserción y búsqueda con una eficiencia promedio de $O(\log n)$. En este sistema la organización empresarial se modela mediante nodos, donde la jerarquía se establece automáticamente a partir del código numérico del empleado por ende el primer registro actuará como raíz y los subordinados se distribuyen en subárboles según la lógica de ordenamiento del BST.

2. Objetivos

2.1 Objetivo General

Implementar un sistema de gestión de personal en C++ mediante una estructura de Árbol Binario de Búsqueda (BST), con el fin de optimizar la organización jerárquica y garantizar la eficiencia en las operaciones de administración de datos.

2.2 Objetivos Específicos

1. Implementar la lógica de inserción de nodos basada en códigos numéricos únicos, garantizando la integridad del ordenamiento BST en cada operación.
2. Programar los algoritmos de recorrido en orden, Preorden y Postorden para ofrecer distintas perspectivas de visualización del organigrama empresarial.
3. Calcular y mostrar características del árbol tales como la altura total y la identificación de nodos hoja

3. Marco Teórico

Para la comprensión y correcta interpretación del sistema implementado se presentan a continuación los conceptos fundamentales de la teoría de árboles binarios de búsqueda también conocido como BST los cuales fueron aplicados al contexto empresarial.

3.1 Árbol Binario de Búsqueda (BST)

Un Árbol Binario de Búsqueda es una estructura de datos dinámica y jerárquica donde cada nodo posee un máximo de dos descendientes. La arquitectura del BST se rige por una propiedad de ordenamiento estricta: para un nodo N , el subárbol izquierdo contiene valores menores a N , y el subárbol derecho valores mayores. Esta característica permite que las búsquedas se realicen mediante un proceso de descartes sucesivos, reduciendo drásticamente el tiempo de respuesta frente a estructuras lineales.

3.2 Componentes del Árbol y su Equivalente Empresarial

Termino Técnico	Definición	Equivalente empresarial
Raiz	Primer nodo del árbol no tiene padre.	Dirección General de la empresa.
Nodo Interno	Nodo que tiene al menos un hijo.	Gerente o supervisor con personal a cargo.
Nodo Hoja	Nodo sin hijos en el árbol.	Empleado operativo sin subordinados.
Altura	Número máximo de niveles desde la raíz hasta el nodo mas profundo.	Numero de niveles jerárquicos en el organigrama.
Subárbol Izquierdo	Conjunto de nodos con clave menor a la raíz.	Empleados con código menor al del nodo actual.
Subárbol Derecho	Conjunto de nodos con clave mayor a la raíz.	Empleados con código mayor al del nodo actual.

3.3 Operaciones Fundamentales del BST

Inserción

La inserción en un BST comienza siempre desde la raíz. El nuevo elemento se compara con el nodo actual, si es menor el algoritmo desciende por el subárbol izquierdo, si es mayor, por el derecho. Este proceso se repite recursivamente hasta encontrar una posición vacía (puntero nulo), donde se crea el nuevo nodo

Búsqueda

La búsqueda aprovecha la misma propiedad de ordenamiento para localizar un elemento eficientemente. Partiendo de la raíz, se compara el valor buscado con el nodo actual y se desciende por la rama correspondiente hasta encontrar el elemento o alcanzar un nodo nulo.

Recorridos

Los recorridos definen el orden en que se visitan los nodos del árbol. Existen tres variantes principales, cada una con una aplicación practica distinta en el sistema:

- **Inorden:** Produce los elementos en orden ascendente según la clave. En el sistema, equivale a un listado de empleados ordenado por código de menor a mayor.
- **Preorden:** Visita primero el nodo padre antes que sus hijos. Es útil para visualizar la jerarquía de mando, mostrando al superior antes que a sus subordinados.
- **Postorden:** Visita los hijos antes que el padre. Es especialmente útil para operaciones de liberación de memoria y destrucción de nodos de forma segura.

Además de las operaciones de manipulación de datos, el sistema implementa funciones de diagnóstico para determinar la topología del árbol:

Obtención de la Raíz

Es la operación más sencilla pero fundamental para validar la jerarquía. Permite confirmar que el primer nodo insertado (el de mayor rango administrativo) se mantiene como el padre central de toda la estructura.

Cálculo de Altura

Esta operación se utiliza principalmente para medir la profundidad máxima del árbol. El algoritmo calcula la altura de los subárboles izquierdo y derecho de forma independiente y retorna el valor máximo más uno

- **Importancia:** Ayuda a definir cuántos niveles de mando o escalas salariales existen desde la dirección hasta el puesto más operativo.

Identificación de Nodos Hoja

A diferencia de los recorridos generales, esta función filtra los nodos basándose en una condición en el que tanto el puntero izquierdo como el derecho sean nulos.

- **Importancia:** Permite listar exclusivamente al personal que no tiene subordinados a su cargo, facilitando el análisis de la base operativa de la empresa.

3.4 Complejidad Algorítmica

En términos de eficiencia, las operaciones de inserción, búsqueda y cálculo de altura en este sistema operan bajo una complejidad de $O(\log n)$ en casos promedio. Es importante destacar que, para mantener este rendimiento se deben evitar inserciones en orden estrictamente secuencial lo que prevendría que el árbol se degrade en una estructura lineal de eficiencia $O(n)$.

4. Arquitectura del Sistema

El sistema se desarrolló bajo el paradigma de Programación Orientada a Objetos (POO) y se organiza en múltiples archivos distribuyéndolo según sus responsabilidades.

Archivo	Responsabilidad
Empleado.h	Define las estructuras de datos: la entidad Empleado (código, nombre, cargo) y el nodo del árbol, que encapsula un Empleado más los punteros a sus hijos.
ArbolBST.h	Declara la clase ArbolBST con sus métodos públicos y privados, incluyendo la referencia a la raíz del árbol.
ArbolBST.cpp	Contiene la implementación completa de todos los métodos de la clase: inserción recursiva, búsqueda, recorridos, cálculo de altura, identificación de hojas y carga de datos de prueba.
main.cpp	Gestiona la interfaz de usuario mediante un menú de opciones y coordina las llamadas a los métodos de ArbolBST.

5. Análisis Técnico del Código

5.1 Archivo Empleado.h

El archivo Empleado.h define dos estructuras de datos fundamentales para el funcionamiento del sistema. El uso de structs es apropiado dado que estas entidades actúan como contenedores de datos sin lógica de negocio propia.

La estructura Empleado agrupa tres atributos los cuales son: “código” de tipo entero, que actúa como clave primaria y determina la posición del nodo en el árbol, el campo “nombre” de tipo string, que almacena el nombre completo del empleado, y finalmente el campo “cargo” de tipo string, que describe la función del empleado dentro de la organización.

La estructura Nodo encapsula un objeto Empleado y dos punteros dinámicos tanto el izquierdo como el derecho, que apuntan a los nodos hijos. El constructor de Nodo inicializa ambos punteros a nullptr,

```

#ifndef EMPLEADO_H
#define EMPLEADO_H

#include <string>
using namespace std;

struct Empleado {
    int codigo;
    string nombre;
    string cargo;
};

struct Nodo {
    Empleado dato;
    Nodo* izquierdo;
    Nodo* derecho;

    Nodo(Empleado emp) {
        dato = emp;
        izquierdo = nullptr;
        derecho = nullptr;
    }
};

#endif

```

5.2 Archivo ArbolBST.h

La clase ArbolBST encapsula todo el comportamiento del árbol.

Los métodos privados (insertar, buscar, inorden, preorden, postorden, altura, mostrarHojas, mostrarEmpleado) implementan la lógica recursiva del árbol. Reciben como parámetro un puntero al nodo actual, lo que les permite operar sobre cualquier subárbol.

Los métodos públicos (insertarEmpleado, buscarEmpleado, mostrarRaiz, mostrarInorden.) constituyen la interfaz del árbol. Actúan como intermediarios que invocan a sus equivalentes privados pasando la raíz como punto de inicio y ocultando los detalles de implementación al usuario de la clase.

```

1  #ifndef ARBOLBST_H
2  #define ARBOLBST_H
3
4  #include "Empleado.h"
5
6  class ArbolBST {
7  private:
8      Nodo* raiz;
9      Nodo* insertar(Nodo* nodo, Empleado emp);
10     Nodo* buscar(Nodo* nodo, int codigo);
11     void inorden(Nodo* nodo);
12     void preorden(Nodo* nodo);
13     void postorden(Nodo* nodo);
14     int altura(Nodo* nodo);
15     void mostrarHojas(Nodo* nodo);
16     void mostrarEmpleado(Nodo* nodo);
17
18 public:
19     ArbolBST();
20     void insertarEmpleado(Empleado emp);
21     void buscarEmpleado(int codigo);
22     void mostrarRaiz();
23     void mostrarInorden();
24     void mostrarPreorden();
25     void mostrarPostorden();
26     void mostrarAltura();
27     void mostrarNodosHoja();
28     void cargarDatosPrueba();
29 };
30
31 #endif

```

5.3 Archivo ArbolBST.cpp

La implementación en ArbolBST.cpp utiliza recursividad, lo que permite tratar cada nodo como un subárbol y simplificar las operaciones.

Las operaciones de inserción y búsqueda siguen la regla del BST por lo que si el valor es menor, se va a la izquierda, si es mayor se dirige a la derecha. Esto mantiene el árbol ordenado y permite una eficiencia de $O(\log n)$.

Los recorridos (inorden, preorden y postorden) visitan los nodos en diferente orden, permitiendo obtener datos organizados o representar la estructura del árbol.

```
1  #include <iostream>
2  #include <algorithm>
3  #include "ArbolBST.h"
4
5  using namespace std;
6
7  ArbolBST::ArbolBST() : raiz(nullptr) {}
8
9  Nodo* ArbolBST::insertar(Nodo* nodo, Empleado emp) {
10     if (nodo == nullptr) return new Nodo(emp);
11     if (emp.codigo < nodo->dato.codigo)
12         nodo->izquierdo = insertar(nodo->izquierdo, emp);
13     else if (emp.codigo > nodo->dato.codigo)
14         nodo->derecho = insertar(nodo->derecho, emp);
15     return nodo;
16 }
17
18 Nodo* ArbolBST::buscar(Nodo* nodo, int codigo) {
19     if (nodo == nullptr || nodo->dato.codigo == codigo) return nodo;
20     if (codigo < nodo->dato.codigo) return buscar(nodo->izquierdo, codigo);
21     return buscar(nodo->derecho, codigo);
22 }
23
24 void ArbolBST::mostrarEmpleado(Nodo* nodo) {
25     cout << "ID: " << nodo->dato.codigo << " | " << nodo->dato.nombre << " | " << nodo->dato.cargo << endl;
26 }
27
28 // Implementación de recorridos
29 void ArbolBST::inorden(Nodo* nodo) {
30     if (nodo) { inorden(nodo->izquierdo); mostrarEmpleado(nodo); inorden(nodo->derecho); }
31 }
32
33 void ArbolBST::preorden(Nodo* nodo) {
34     if (nodo) { mostrarEmpleado(nodo); preorden(nodo->izquierdo); preorden(nodo->derecho); }
35 }
36
37 void ArbolBST::postorden(Nodo* nodo) {
38     if (nodo) { postorden(nodo->izquierdo); postorden(nodo->derecho); mostrarEmpleado(nodo); }
39 }
40
41 int ArbolBST::altura(Nodo* nodo) {
42     if (!nodo) return 0;
43     return 1 + max(altura(nodo->izquierdo), altura(nodo->derecho));
44 }
45
46 void ArbolBST::mostrarHojas(Nodo* nodo) {
47     if (!nodo) return;
48     if (!nodo->izquierdo && !nodo->derecho) mostrarEmpleado(nodo);
49     mostrarHojas(nodo->izquierdo);
50     mostrarHojas(nodo->derecho);
51 }
52
53 // Métodos públicos que llaman a los privados
54 void ArbolBST::insertarEmpleado(Empleado emp) { raiz = insertar(raiz, emp); }
55 void ArbolBST::mostrarInorden() { inorden(raiz); }
56 void ArbolBST::mostrarPreorden() { preorden(raiz); }
57 void ArbolBST::mostrarPostorden() { postorden(raiz); }
58 void ArbolBST::mostrarAltura() { cout << "Altura: " << altura(raiz) << endl; }
59 void ArbolBST::mostrarNodosHoja() { mostrarHojas(raiz); }
60 void ArbolBST::mostrarRaiz() { if (raiz) mostrarEmpleado(raiz); }
61
62 void ArbolBST::buscarEmpleado(int codigo) {
63     Nodo* res = buscar(raiz, codigo);
64     if (res) mostrarEmpleado(res);
65     else cout << "No encontrado." << endl;
66 }
```

Se implementó una función de carga automática de datos que permite construir un árbol binario de búsqueda. A partir de los valores ingresados, se genera una estructura jerárquica donde la raíz representa la empresa, los

nodos intermedios a los gerentes y los nodos hoja al personal operativo, permitiendo validar el correcto funcionamiento del árbol en términos de organización, recorrido y análisis estructural.

```
void ArbolBST::cargarDatosPrueba() {
    insertarEmpleado({50, "CEO Principal", "Raiz"});
    insertarEmpleado({30, "Gerente IT", "Interno"});
    insertarEmpleado({70, "Gerente RRHH", "Interno"});
    insertarEmpleado({20, "Analista 1", "Hoja"});
    insertarEmpleado({40, "Analista 2", "Hoja"});
    insertarEmpleado({60, "Asistente 1", "Hoja"});
    insertarEmpleado({80, "Asistente 2", "Hoja"});
}
```

5.4 Archivo main.cpp

El archivo principal contiene la lógica que permite interactuar con el sistema mediante un menú, desde el cual el usuario puede ejecutar diferentes operaciones sobre el árbol binario de búsqueda, como insertar, buscar y visualizar información, facilitando así el uso y prueba del programa.

```
#include <iostream>
#include "ArbolBST.h"

using namespace std;

int main() {
    ArbolBST arbol;
    int opcion, cod;

    do {
        cout << "\n===== MENU ARBOL BST EMPRESARIAL =====\n";
        cout << "1. Insertar empleado\n";
        cout << "2. Buscar empleado\n";
        cout << "3. Mostrar raiz del arbol\n";
        cout << "4. Recorrido en orden\n";
        cout << "5. Recorrido preorden\n";
        cout << "6. Recorrido postorden\n";
        cout << "7. Mostrar altura\n";
        cout << "8. Mostrar hojas\n";
        cout << "9. Cargar datos de prueba\n";
        cout << "0. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        switch(opcion) {
            case 1: {
                Empleado e;
                cout << "Codigo: "; cin >> e.codigo;
                cout << "Nombre: "; cin.ignore(); getline(cin, e.nombre);
                cout << "Cargo: "; getline(cin, e.cargo);
                arbol.insertarEmpleado(e);
                break;
            }
            case 2: {
                cout << "Ingrese codigo a buscar: ";
                cin >> cod;
                arbol.buscarEmpleado(cod);
                break;
            }
            case 3: {
                arbol.mostrarRaiz();
                break;
            }
            case 4: {
                arbol.mostrarInorden();
                break;
            }
            case 5: {
                arbol.mostrarPreorden();
                break;
            }
            case 6: {
                arbol.mostrarPostorden();
                break;
            }
            case 7: {
                arbol.mostrarAltura();
                break;
            }
            case 8: {
                arbol.mostrarNodosHoja();
                break;
            }
            case 9: {
                arbol.cargarDatosPrueba();
                break;
            }
            case 0: {
                cout << "Saliendo del sistema...\n";
                break;
            }
            default: {
                cout << "Opcion no valida.\n";
            }
        }
    } while (opcion != 0);

    return 0;
}
```

6. Pruebas y Resultados

Para la validación del sistema, se realizó un conjunto de pruebas sistemáticas utilizando el conjunto de datos de prueba incorporado en la opción 9 del menú. A continuación, se documentan los resultados esperados para cada funcionalidad.

6.1 Estructura del Árbol de Prueba

Al cargar los datos de prueba, el sistema inserta los siguientes empleados en orden, construyendo el árbol cuya estructura se detalla a continuación:

Código	Nombre	Cargo	Posición en el Árbol
50	Empresa UTA	Raiz	Raiz del árbol
30	Gerente Ventas	Nodo Interno	Hijo izquierdo de 50
70	Gerente Finanzas	Nodo Interno	Hijo derecho de 50
20	Analista 1	Hoja	Hijo izquierdo de 30
40	Analista 2	Hoja	Hijo derecho de 30
60	Asistente 1	Hoja	Hijo izquierdo de 70
80	Asistente 2	Hoja	Hijo derecho de 70

6.2 Verificación de Recorridos

Los resultados esperados para cada tipo de recorrido sobre el árbol de prueba son los siguientes:

Recorrido	Secuencia de Códigos	Interpretación
Inorden	20 -> 30 -> 40 -> 50 -> 60 -> 70 -> 80	Empleados ordenados de menor a mayor código.
Preorden	50 -> 30 -> 20 -> 40 -> 70 -> 60 -> 80	Jefe antes que subordinados.
Postorden	20 -> 40 -> 30 -> 60 -> 80 -> 70 -> 50	Subordinados antes que el jefe.

- Ejecución Recorrido Inorden

```
ID: 20 | Analista 1 (Hoja)
ID: 30 | Gerente IT (Interno)
ID: 40 | Analista 2 (Hoja)
ID: 50 | CEO Principal (Raiz)
ID: 60 | Asistente 1 (Hoja)
ID: 70 | Gerente RRHH (Interno)
ID: 80 | Asistente 2 (Hoja)
```

- Ejecución Recorrido Preorden


```
ID: 50 | CEO Principal (Raiz)
ID: 30 | Gerente IT (Interno)
ID: 20 | Analista 1 (Hoja)
ID: 40 | Analista 2 (Hoja)
ID: 70 | Gerente RRHH (Interno)
ID: 60 | Asistente 1 (Hoja)
ID: 80 | Asistente 2 (Hoja)
```

- Ejecución Recorrido Postorden

```
ID: 20 | Analista 1 (Hoja)
ID: 40 | Analista 2 (Hoja)
ID: 30 | Gerente IT (Interno)
ID: 60 | Asistente 1 (Hoja)
ID: 80 | Asistente 2 (Hoja)
ID: 70 | Gerente RRHH (Interno)
ID: 50 | CEO Principal (Raiz)
```

6.3 Métricas Estructurales del Árbol

Mediante el uso de los datos de prueba, se validaron las métricas de la topología del árbol. Los resultados esperados y verificados en el sistema son los siguientes:

- **Raíz:** Identificación del nodo superior como el primer registro ingresado (Código 50).

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 3
ID: 50 | CEO Principal (Raiz)
```

- **Altura:** Verificación de la profundidad máxima del organigrama, alcanzando un total de 3 niveles.

```

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 7
Altura: 3

```

- **Nodos Hoja:** Localización de los empleados terminales que no poseen subordinados en la estructura (Códigos 20, 40, 60 y 80) .

```

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 8
ID: 20 | Analista 1 (Hoja)
ID: 40 | Analista 2 (Hoja)
ID: 60 | Asistente 1 (Hoja)
ID: 80 | Asistente 2 (Hoja)

```

6.4 Verificación de Búsqueda

Se ejecutaron pruebas de consulta para verificar la eficiencia del algoritmo de búsqueda en diferentes escenarios:

- **Búsqueda Exitosa:** Localización inmediata de un empleado existente mediante su código numérico.
- **Búsqueda Fallida:** Validación del manejo de excepciones cuando el código ingresado no consta en la base de dato

```

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 2
Ingrese codigo a buscar: 50
ID: 50 | CEO Principal (Raiz)

```

```

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 2
Ingrese codigo a buscar: 55
No encontrado.

```

6.5 Inserción Manual de Empleados

Se comprobó la capacidad del sistema para integrar nuevos registros en tiempo de ejecución de forma dinámica. El programa asegura que cada nuevo empleado se ubique en la posición jerárquica correcta, manteniendo en todo momento la integridad y el ordenamiento de la estructura BST.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 1
Codigo: 30
Nombre: DAVID
Cargo: EMPLEADO 3

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 1
Codigo: 90
Nombre: JORJE
Cargo: EMPLEADO 4

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 1
Codigo: 80
Nombre: MIRIAM
Cargo: EMPLEADA 1

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 1
Codigo: 70
Nombre: JAVIER
Cargo: EMPLEADO 2

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 1
Codigo: 40
Nombre: ESTIVEN
Cargo: GERENTE FINANZAS

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 1
Codigo: 60
Nombre: KEVIN
Cargo: GERENTE VENTAS

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz del arbol
4. Recorrido en orden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
9. Cargar datos de prueba
0. Salir
Seleccione una opcion: 1
Codigo: 50
Nombre: uta
Cargo: EMPRESA
```

7. Conclusiones

A partir del desarrollo, implementación y validación del sistema, se derivan las siguientes conclusiones:

1. Se implementó la lógica de inserción de nodos basada en códigos numéricos únicos, garantizando la integridad del ordenamiento BST en cada operación.
2. Se programó los algoritmos de recorrido en orden, Preorden y Postorden para ofrecer distintas perspectivas de visualización del organigrama empresarial.
3. Se calculó y se mostró características del árbol tales como la altura total y la identificación de nodos hoja

8. Referencias Bibliográficas

[1] R. Sedgewick y K. Wayne, "Binary Search Trees," *Algorithms*, 4ta ed. Princeton University, 2022. [En línea]. Disponible en: <https://algs4.cs.princeton.edu/32bst/> [Accedido: 26-abr-2026].

[2] FreeCodeCamp, "Binary Search Trees (BST) Explained with Examples," *freeCodeCamp.org*, 2021. [En línea]. Disponible en: <https://www.freecodecamp.org/news/binary-search-trees-bst-explained-with-examples/> [Accedido: 26-abr-2026].

[3] SlideShare, "Árboles binarios de búsqueda: Operaciones," *SlideShare*, 2024. [En línea]. Disponible en: <https://es.slideshare.net/slideshow/arboles-binarios-de-busqueda-operaciones/267363941> [Accedido: 26-abr-2026].

9. Anexos

Link GitHub

<https://github.com/Estiven453/Arbol-BST-Empresa-cpp.git>